

Computer Organization and Architecture

Introduction to Instruction Pipelining and Hazards

April 2026

Agenda

- Instruction level parallelism
- Instruction Pipelining for RISC
- Pipelining Hazards
 - Structural Hazards
 - Data Hazards
 - Control Hazards
- Numerical Examples

Pipelining Hazards

- Hazards are circumstances circumstances that would cause incorrect execution if next instruction is fetched and executed
- Pipelining Hazards
 - Structural Hazards
 - Data Hazards
 - Control Hazards

Pipelining

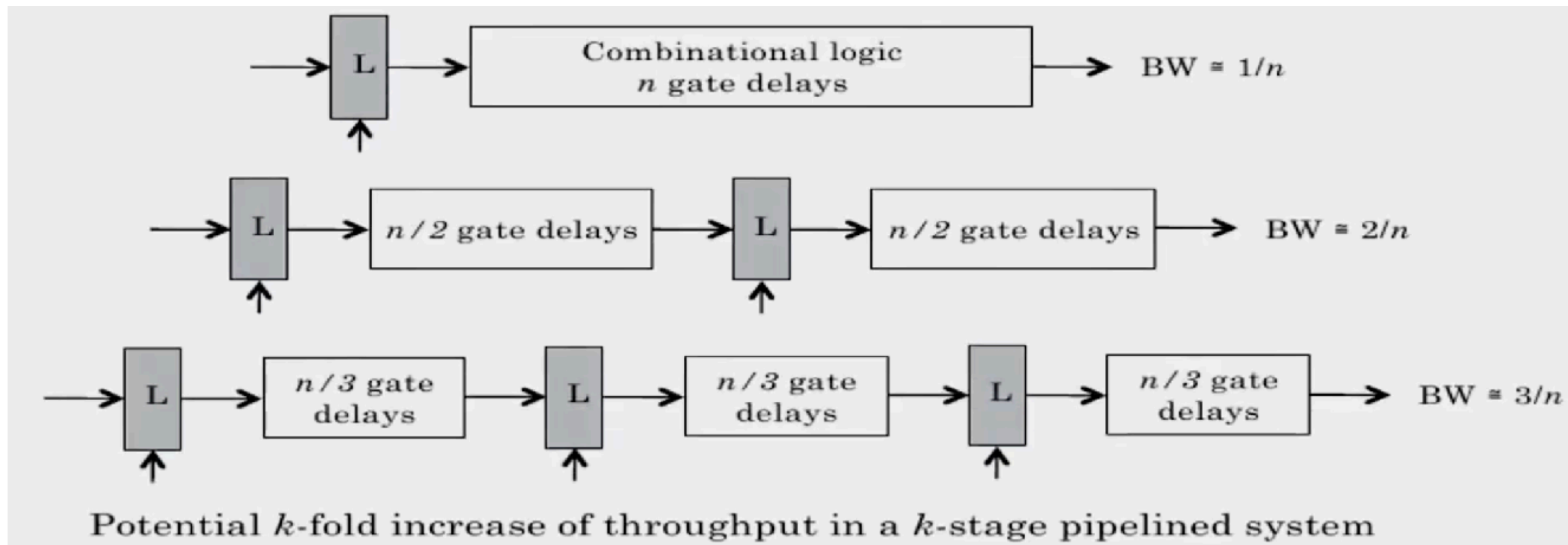
- Instruction level parallelism (ILP): parallelism in instruction execution within a program.
- Pipelining exploits ILP by allowing different instructions to be in various stages of execution concurrently.

Pipelining characteristics

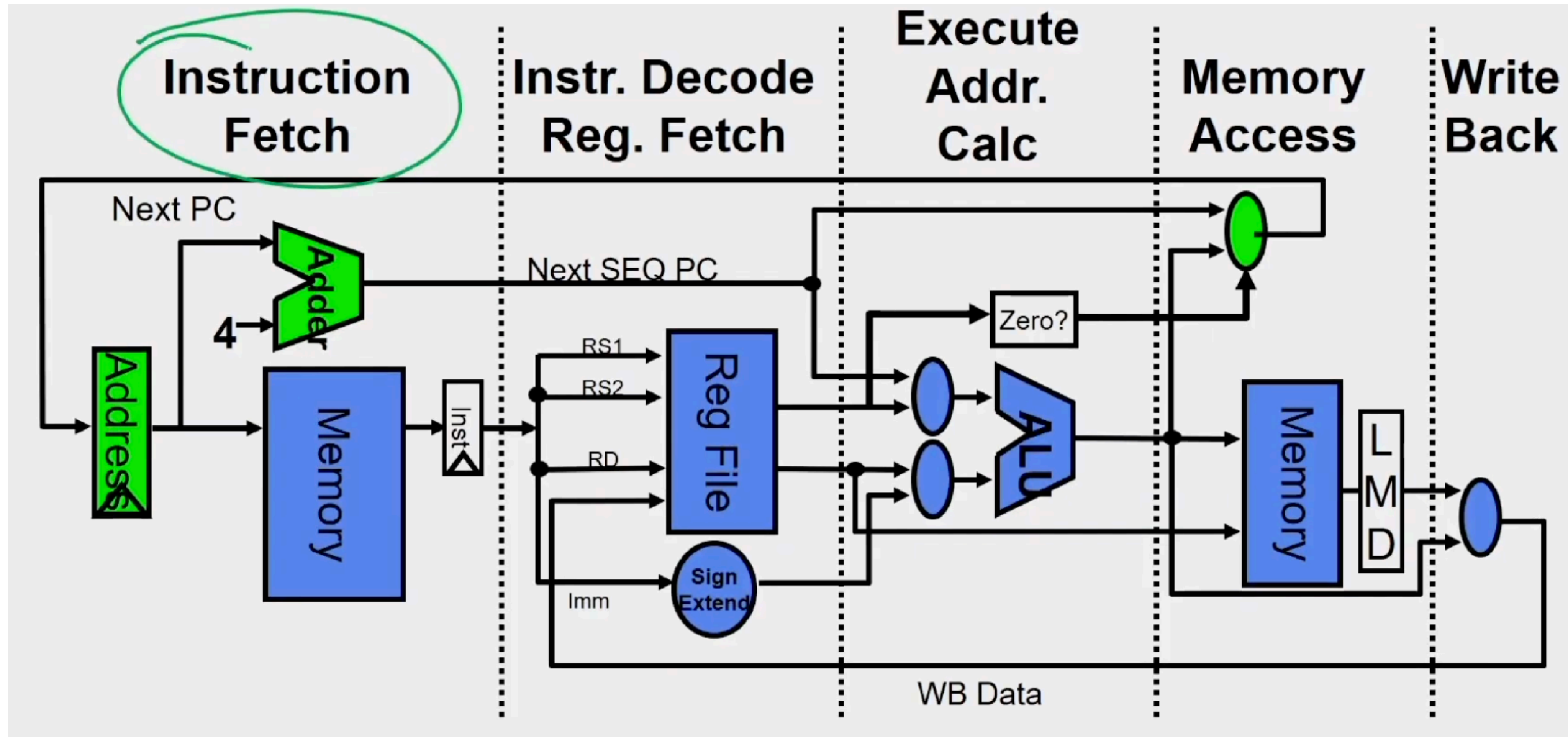
- Pipelining doesn't reduce latency of single task, it improves throughput of entire workload
- Pipeline rate limited by slowest pipeline stage
- Potential speedup = Number of pipe stages
- Unbalanced lengths of pipe stages reduces speedup
- Time to fill pipeline and time to drain it reduces speedup

Pipelining in circuits

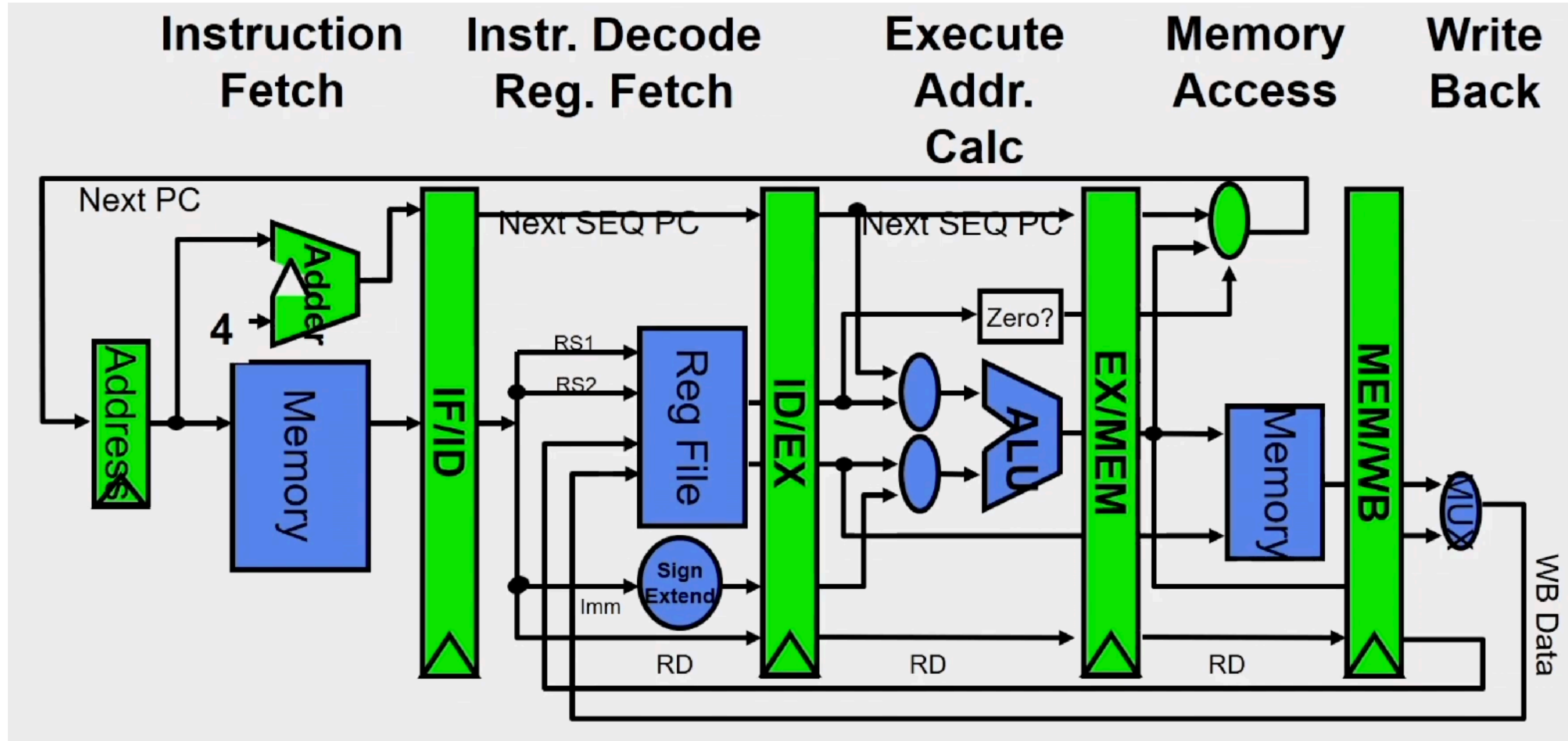
- Pipelining partitions the system into multiple independent stages with added buffers between the stages.
- Pipelining can increase the throughput of a system.



Un-Pipelined RISC System



Pipelined RISC System



Numerical Example 1

Pipeline Design

- The time delay of various segments in a 4-stage pipeline are $t_1=35$ ns, $t_2 = 30$ ns, $t_3 = 40$ ns, $t_4=45$ ns. The interface register delay time is $t=5$ ns. How long would it take to complete 100 instructions in the pipeline ? (Assume there is no dependency between the instructions).

Numerical Example 1

Pipeline Design

- The time delay of various segments in a 4-stage pipeline are $t_1=35$ ns, $t_2 = 30$ ns, $t_3 = 40$ ns, $t_4=45$ ns. The interface register delay time is $t=5$ ns. How long would it take to complete 100 instructions in the pipeline ? (Assume there is no dependency between the instructions).
- Avg. time taken per one instruction using the pipeline: $\text{Max } \{t_{\text{stage}}\} + 5 \text{ ns} = 45 \text{ ns} + 5 \text{ ns} = 50 \text{ ns}$ per instruction on average.

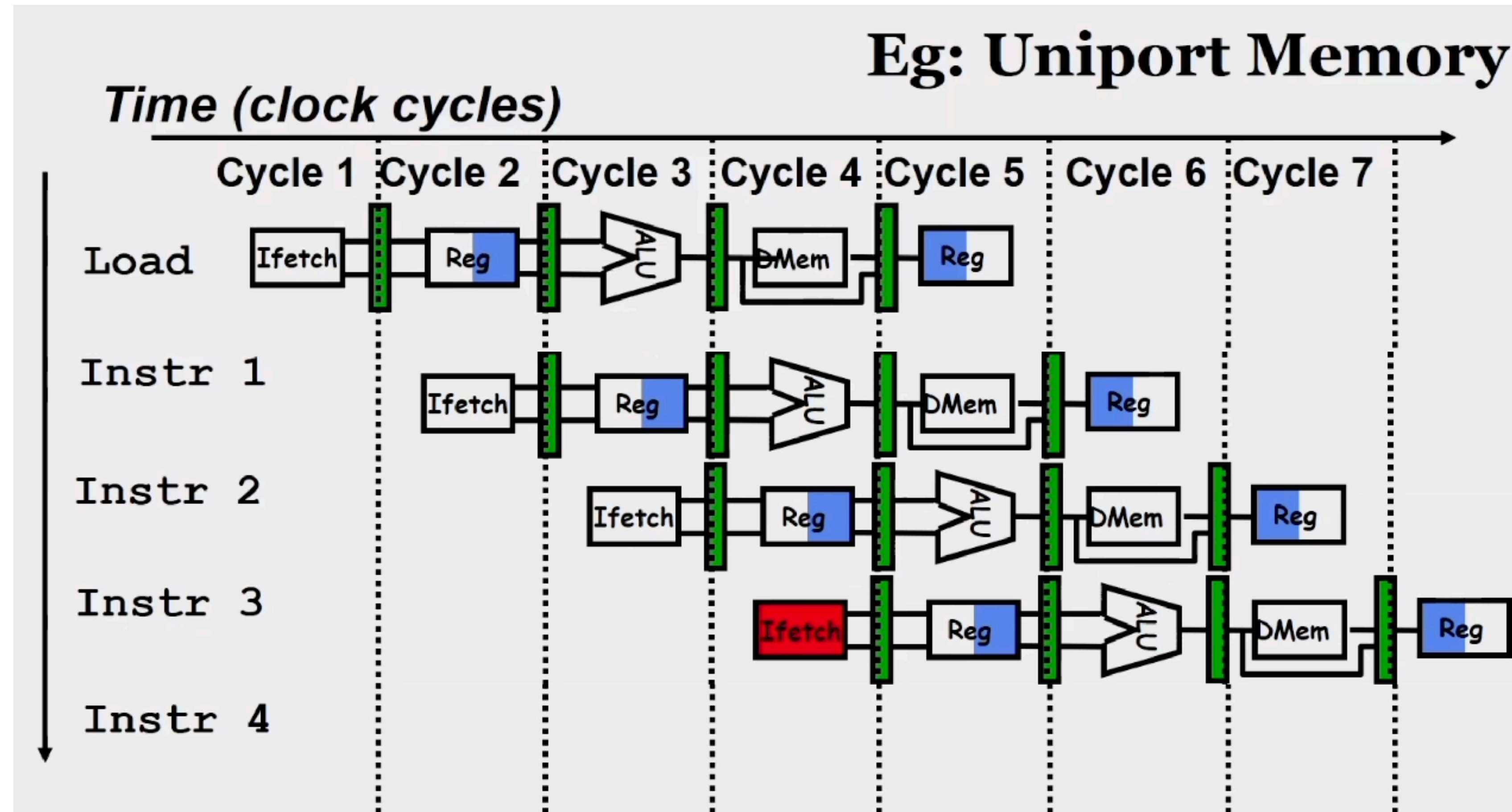
Numerical Example 1

Pipeline Design

- The time delay of various segments in a 4-stage pipeline are $t_1=35$ ns, $t_2 = 30$ ns, $t_3 = 40$ ns, $t_4=45$ ns. The interface register delay time is $t=5$ ns. How long would it take to complete 100 instructions in the pipeline ? (Assume there is no dependency between the instructions).
- Pipeline Cycle uses a clock with period of maximum time delay among the stages.
- Avg. time taken per one pipeline cycle: $\text{Max } \{t_{\text{stage}}\} + 5 \text{ ns} = 45 \text{ ns} + 5 \text{ ns} = 50 \text{ ns}$ per instruction on average.
- Each state is completed in a clock cycle of period 50 ns. The first instruction takes 4 clockcycles to complete. After first 4 clockcycles, one instruction gets completed in each clock cycle. For 100 instructions, we need $4 + 99$ clockcycles $= 103 \times 50 = 5150$ ns

Structural Hazards

- **Structural hazards:**
Different instructions, at different stages, in the pipeline want to use the same hardware resource
- In the figure, in cycle 4, Instrc 3 is trying to access the memory in fetch stage while Load instruction is in memory access stage. But the memory is uniport, i.e. allows only one simultaneous access.

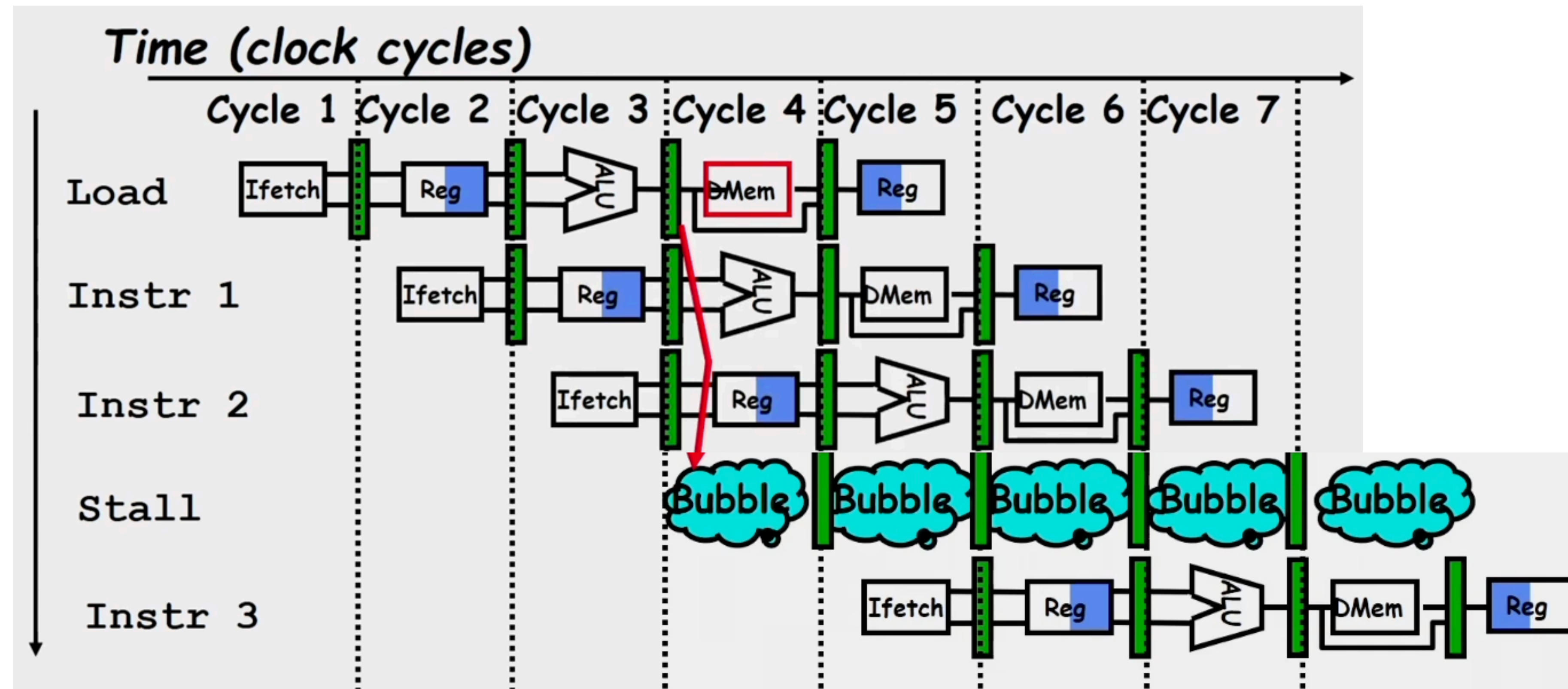


Structural Hazard Solution

- Eliminate the use same hardware for two different things at the same time
- Solution 1: Wait
 - must detect the hazard
 - must have a mechanism to stall
- Solution 2: Duplicate hardware
 - Multiple such units will help both instruction to progress

Structural Hazard Solution

- Eliminate the use same hardware for two different things at the same time
- Solution 1: Wait
 - must detect the hazard
 - must have a mechanism to stall
- Solution 2: Duplicate hardware
 - Multiple such units will help both instruction to progress

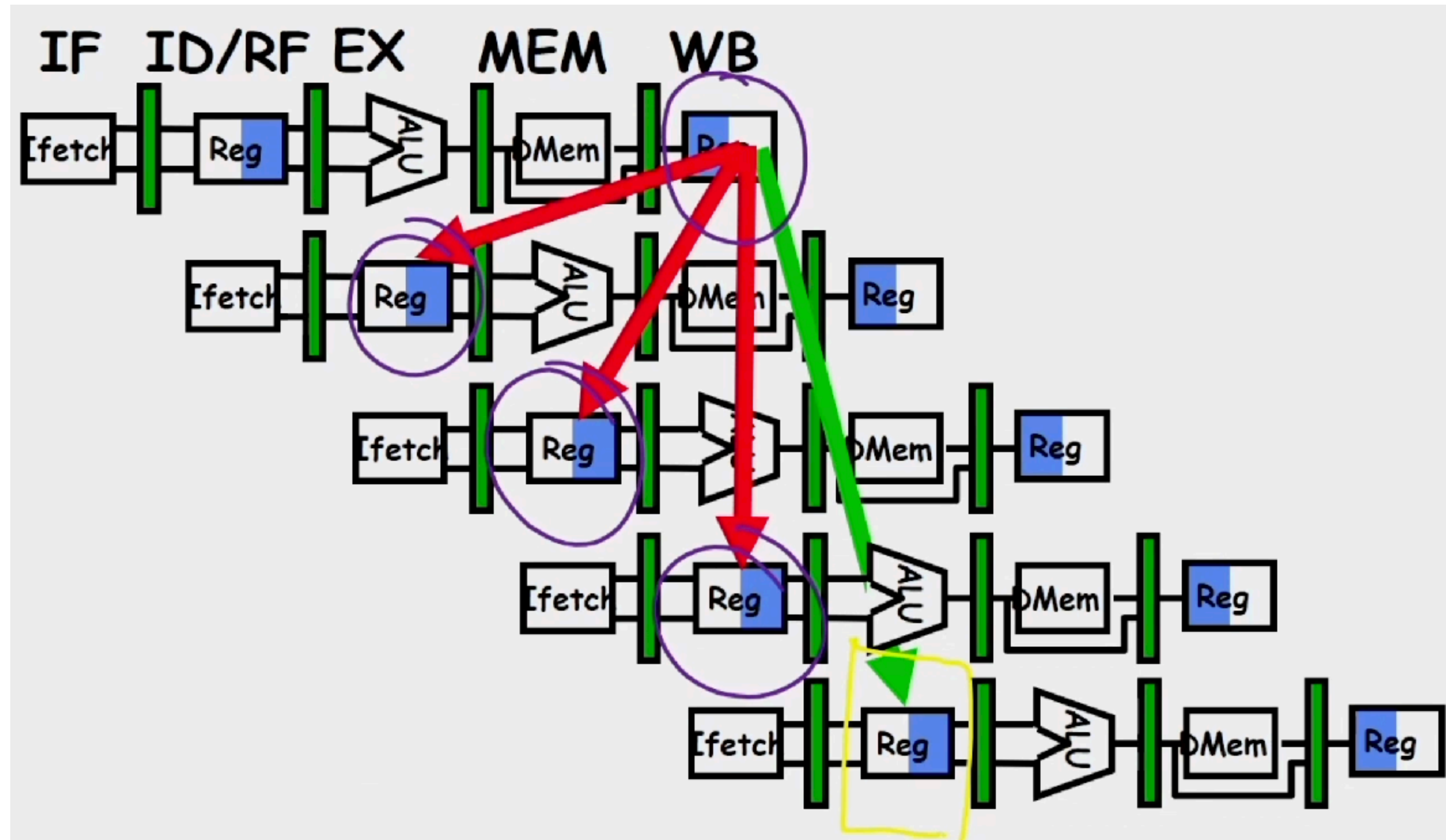


Data Hazards

- **Data hazards:** An instruction in the pipeline requires data to be computed by a previous instruction still in the pipeline
- RAW hazards
- WAR hazards
- WAW hazards

Data Hazards

- add r1, r2, r3
- sub r4, r1, r3
- and r6, r1, r7
- or r8, r1, r9
- xor r10, r1, r11



Data Hazards

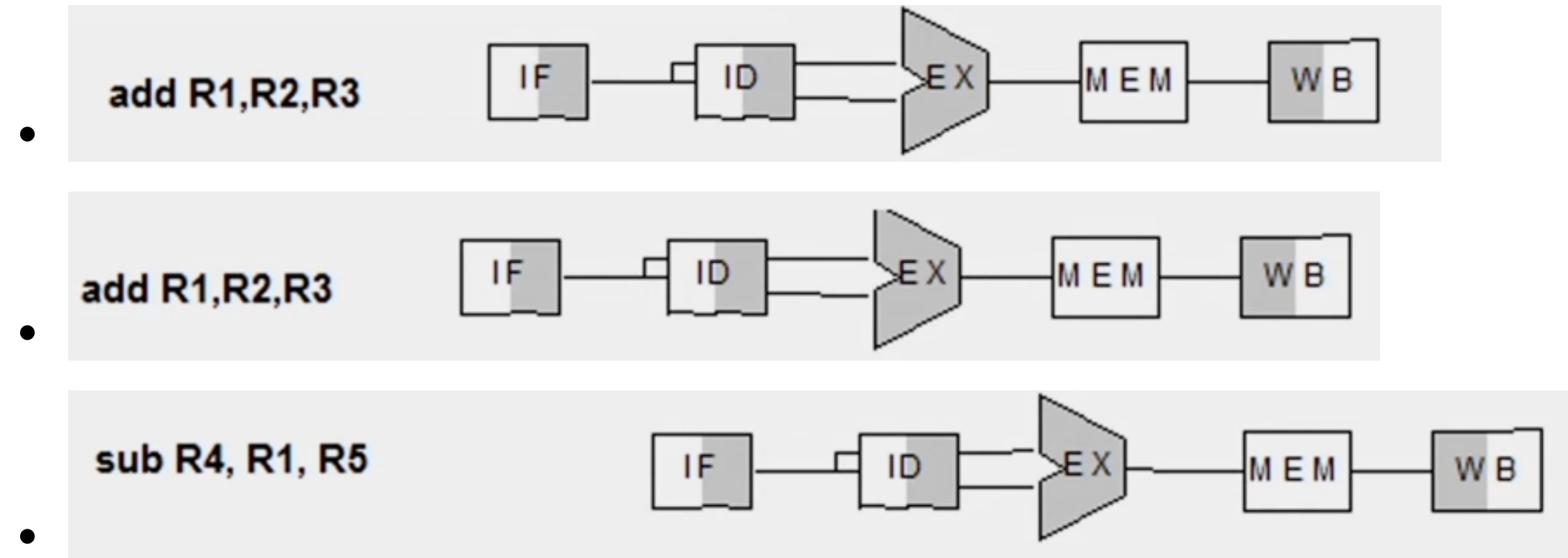
- RAW - Read After Write Hazard:
- I1: Add r1, r2, r3
- I2: Mul r5, r1, r4
- Here, I2 attempts to read r1 before I1 writes back to r1
- Caused by data dependence between nearby instructions

Data Hazards

- WAR - Write After Read Hazard:
- I1: Add r4, r1, r3
- I2: Mul r1, r2, r3
- I3: Mul r6, r1, r7
- If there is a violation in the order of I1 and I2, then I3 reads the wrong value
- This hazard cannot happen in the 5 stage inorder pipeline we discussed earlier
-

Operand forwarding

- Data hazard: instruction needs data from the result of a previous instruction still executing in pipeline
- Forward data between the stages of instructions if possible.

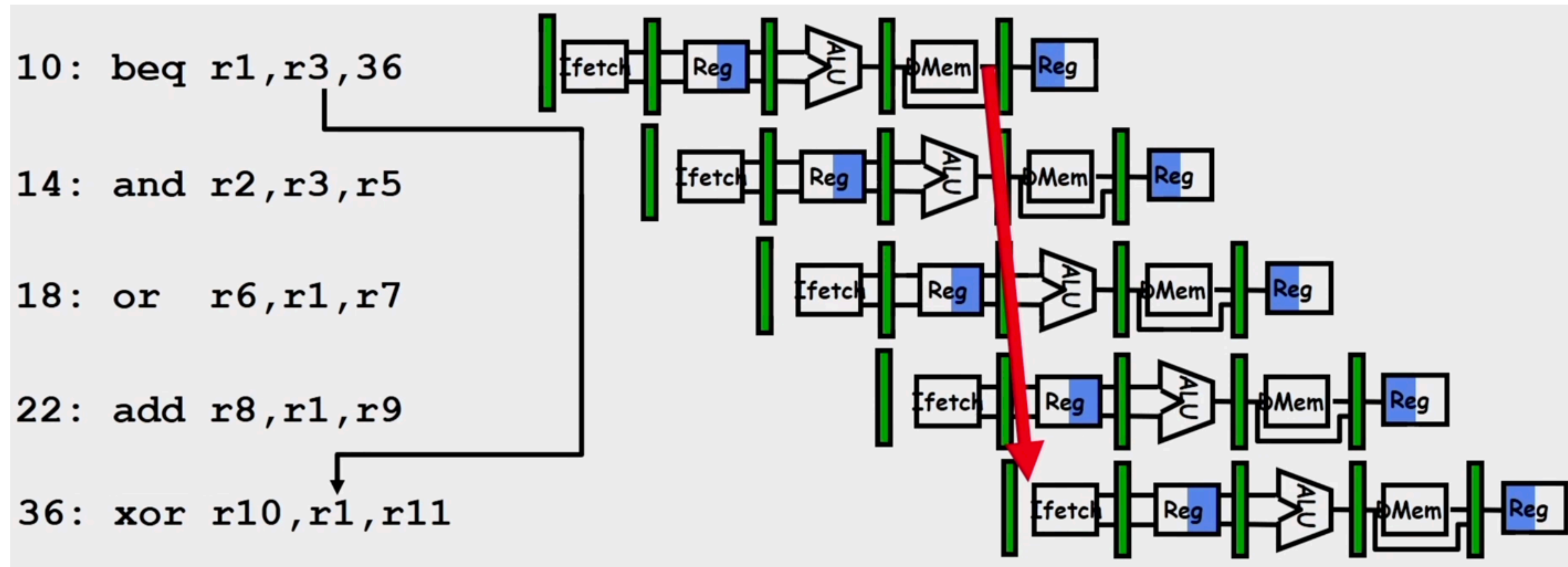


- Here, the output of EX stage of Instr 2 can be forwarded to the input of EX stage of Instr 3
- This way of handling data hazard is called **Operand forwarding**
- **Operand forwarding cannot resolve all data hazards**
- **Sometimes Stalling is needed.**

Data Hazards

- WAW - Write After Write Hazard:
- I: sub r1, r4, r3
- J: add r1, r2, r3
- K: mul r6, r1, r7
- Here, Inst. J writes operand before Inst. K, writes it.
-

Control Hazards on Branches



- **Control hazards:** Succeeding instruction, to put into pipeline, depends on the outcome of a previous branch instruction, already in pipeline

Numerical Example 1

Pipelining Hazards

- Consider the sequence of instructions below and identify which one of the following statements is true. [r1-r6 are the CPU registers]
 - add r4, r2, r3
 - sub r4, r5, r3
 - mul r5, 14, r2
 - add r6, r5, r1
- Soln: There is a WAW hazard in r4 (for instructions 1 and 2).
- RAW hazard in r4 (for instructions 2 and 3),
- WAR hazard in r5 (for instructions 2 and 3),
- RAW hazard in r5 (for instructions 3 and 4).

Numerical Example 2

Pipelining Hazards

- Given a non-pipelined architecture running at 1GHz, that takes 5 cycles to finish an instruction. It is later converted to a 5 stage pipeline operating at 800MHz. A stall of 70 cycles happens in 2% of the memory instructions and a stall of 2 cycles happens in 20% of the branch instructions. Suppose, out of the all instructions, 30% are memory instructions and 20% are branch instructions. What is the actual speedup obtained by pipelining?

Numerical Example 2

Pipelining Hazards

- Given a non-pipelined architecture running at 1GHz, that takes 5 cycles to finish an instruction. It is later converted to a 5 stage pipeline operating at 800MHz. A stall of 70 cycles happens in 2% of the memory instructions and a stall of 2 cycles happens in 20% of the branch instructions. Suppose, out of the all instructions, 30% are memory instructions and 20% are branch instructions. What is the actual speedup obtained by pipelining?
- For unpipelined system, $CPI = 5$. Cycle time = $1/1\text{GHz} = 1 \text{ ns}$.
- For pipelined system, $CPI = \text{Base CPI} + \text{Stall CPI}$. Cycle time = $1/800\text{MHz} = 1.25 \text{ ns}$
- Base CPI is 1 (ideally).
- Stall CPI = Average of Branch Stall + Memory Stall

Numerical Example 2

Pipelining Hazards

- Given a non-pipelined architecture running at 1GHz, that takes 5 cycles to finish an instruction. It is later converted to a 5 stage pipeline operating at 800MHz. A stall of 70 cycles happens in 2% of the memory instructions and a stall of 2 cycles happens in 20% of the branch instructions. Suppose, out of the all instructions, 30% are memory instructions and 20% are branch instructions. What is the actual speedup obtained by pipelining?
- For unpipelined system, $CPI = 5$. Cycle time = $1/1GHz = 1 \text{ ns}$.
- For pipelined system, $CPI = \text{Base CPI} + \text{Stall CPI}$. Cycle time = $1/800MHz = 1.25 \text{ ns}$
- Base CPI is 1 (ideally).
- Stall CPI = Average of Branch Stall + Memory Stall = $(0.3 \times 0.02 \times 70) + (0.2 \times 0.2 \times 2) = 0.42 + 0.08 = 0.5$
- CPI of pipelined system = $1 + 0.5 = 1.5$

Numerical Example 2

Pipelining Hazards

- Given a non-pipelined architecture running at 1GHz, that takes 5 cycles to finish an instruction. It is later converted to a 5 stage pipeline operating at 800MHz. A stall of 70 cycles happens in 2% of the memory instructions and a stall of 2 cycles happens in 20% of the branch instructions. Suppose, out of the all instructions, 30% are memory instructions and 20% are branch instructions. What is the actual speedup obtained by pipelining?
- $\text{Speedup} := \text{Exec. time of unpipelined system} / \text{Exec. time of pipelined system}$
- $\text{Exec. time} = \text{CPI} \times \text{cycle time.}$
- $\text{Speedup} = (5 \times 1) / (1.5 \times 1.25) = 2.667$

Numerical Example 3

Pipelining Hazards

- Given a non-pipelined architecture running at 1.5 GHz, that takes 5 cycles to finish an instruction. You want to make it pipelined with 5 stages. Due to hardware overhead the pipelined design will operate only at 1 GHz. 5% of memory instructions cause a stall of 50 cycles, (C 30% of branch instruction cause a stall of 2 cycles and load-ALU combinations cause a stall of 1 cycle. Assume that in a given program, there exist 20% of branch instructions and 30% of memory instructions. 10% of instructions are load-ALU combinations. What is " the speedup of pipelined design over the non-pipelined design?

Numerical Example 3

Pipelining Hazards

- $CPI_{unpipelined} = 5$.
- Clock Cycle time (CCT) unpipelined = $1/1.5\text{GHz} \rightarrow 0.67 \text{ ns}$
- Cycle time pipelined = $1 \text{ GHz} \rightarrow 1 \text{ ns}$
- $Exec_Time_{unpipelined} = CPI \times CCT = 5 \times 0.66 \text{ ns} = 3.33 \text{ ns / instruction}$
- $Effective\ CPI_{pipelined} = Base\ CPI + stall\ CPI$
- $\{stalls = Memory\ stalls + Branch\ stalls + Load - ALU\ stalls\} = 1 + (0.3 \times 0.05 \times 50) + (0.2 \times 0.3 \times 2) + (0.1 \times 1) = 1.97$
- $Exec.\ Time\ for\ pipelined\ system = CP_{pipelined} \times CCT_{pipelined} = 1.97 \text{ ns}$

Numerical Example 4

Pipelining Hazards - Operand forwarding

- A program has 2000 instructions in the sequence Load, ADD, Load, ADD,..... Load, ADD. The ADD instruction depends on the Load instruction right before it. The Load instruction depends on the ADD instruction right before it. If the program is executed on the 5-stage pipeline what would be the actual CPI with and without operand forwarding technique?

Numerical Example 4

Pipelining Hazards - Operand forwarding

- A program has 2000 instructions in the sequence Load, ADD, LOAD, ADD,..... LOAD, ADD. The ADD instruction depends on the LOAD instruction right before it. The LOAD instruction depends on the ADD instruction right before it. If the program is executed on the 5-stage pipeline what would be the actual CPI with and without operand forwarding technique?
- Soln: Consider the following instruction cycle : IF (Fetch) -> ID (Decode) -> EX (Execute) -> MEM (Memory Access)-> WB (Write Back)
- Here, we have an instruction sequence, LOAD, ADD, LOAD, ADD, ...
- Without Operand Forwarding: For LOAD followed by ADD, the ID stage of ADD instruction has to wait for WB of the previous LOAD instruction and so on.
- ID of nth instruction can be only after WB of (n-1)th instruction.

Numerical Example 4

Pipelining Hazards - Operand forwarding

- **Without Operand Forwarding:** For LOAD followed by ADD, the ID part of ADD instruction has to wait for WB of the previous LOAD instruction and so on.
- ID of nth instruction can be only after WB of (n-1)th instruction.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
L.D	IF	ID	EX	ME	WB									
ADD		IF	*	*	*	ID	EX	ME	WB					
L.D						IF	*	*	*	ID	EX	ME	WB	
ADD										IF	*	*	*	ID

- Instructions reach WB at clock cycles 5, 9, 13, 17, 21, 25, 29,
- Last instruction (ADD) reaches WB in $5 + (1999 \times 4) = 8001$ cycles.
- $CPI = 8001 \text{ cycles for } 2000 \text{ instructions} = 4.0005$

Numerical Example 4

Pipelining Hazards - Operand forwarding

- **With Operand Forwarding:** ADD after LOAD has a stall. But Load after ADD does not have a stall.
- Operand forwarding happens from ME stage of LOAD to EX stage of the following ADD. A Stall happens (at t=3, 6, ...)
- Operand forwarding also happens from EX stage of ADD to EX stage of the following LOAD. No stall needed.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
L.D	IF	ID	EX	ME	WB									
ADD		IF	*	ID	EX	ME	WB							
L.D				IF	ID	EX	ME	WB						
ADD					IF	*	ID	EX	ME	WB				

- LOAD instructions reach WB stage at 5, 8, 11, 14 ...
- ADD instructions reach WB stage at 7, 10, 13, 16 ...
- Last ADD instruction reaches WB stage at the following cycle: 7 (first ADD) + 999 x 3 = cycle 3004
- $CPI = 3004/2000 = 1.502$